FinMentor — project documentation

Product: FinMentor

What it is: A private personal-finance Q&A agent. You log in, upload your own statements (PDF/images/text), and ask questions in a chat. Answers come only from chunks of those files. If nothing in the vault is similar enough, the app refuses instead of guessing.

Not: A bank, tax filer, investment advisor, or a chatbot that uses general web knowledge for your money.

This document matches the code in this repository (October 2026): FastAPI backend, static chat UI, optional demo mode, user-scoped Postgres + pgvector, chat history, and a 19-type document vault plus Custom.

---

1. What problem it solves

People keep money facts in PDFs and photos: bank statements, Form 16, insurance, loans, credit cards. Asking “how much did I spend on groceries?” or “what does my policy exclude?” usually means opening files and hunting.

Generic chatbots fail here in two ways:

1. They **do not have the file**. They invent a number.
2. Even with an attached file, they often **mix training knowledge with the document**, so a salary figure or a coverage amount can be wrong with no warning.

FinMentor’s job is narrower: **retrieve your text, then answer, or say it does not know.**

It is built so a user can:

- Keep documents in a personal vault (not a shared pile).
- Ask in chat, including by voice on supporting browsers.
- See which files grounded the answer.
- Get a fixed refusal when similarity is too low (hallucination gate).

---

2. Why it was built this way

Decision	Why
RAG, not “upload then dump the whole PDF into the LLM”	Long statements overflow context; chunk + vector search keeps only the relevant paragraphs.
Hard similarity threshold (`0.75`) before any LLM call	If retrieval is weak, the model must not fill gaps from world knowledge.
User-scoped `documents` / `document_chunks` / `conversations`	One vault per account; retrieval always filters `user_id` .
OCR (Mistral) for images and scanned PDFs	Bank apps and insurers still ship photos and scans.
Configurable embedding + LLM providers	Swap OpenAI / Gemini / Anthropic without rewriting the pipeline.

Decision	Why
Fail-fast config	Missing `SUPABASE_*`, OCR, LLM, or embedding keys crash at startup instead of serving half-broken answers.
Static HTML/CSS/JS	One FastAPI process can serve UI + API (including Render).
Demo auth + in-memory store	The UI and eval can run without a live Supabase project or paid keys. **Demo answers are keyword-grounded, not embedding-grounded.**

---

3. How it was built (pipeline)

End-to-end path after login:

```
 Upload file
   → optional Mistral OCR (image / PDF)
   → store documents row (status: processing)
   → background thread:
         RecursiveCharacterTextSplitter (1000 / 200)
         embeddings (OpenAI 1536-d or Gemini 3072-d truncated to 1536 + L2)
         insert document_chunks
         status: indexed  (or failed + chunks deleted)
Ask question
   → embed query (same model as chunks)
   → match_document_chunks RPC (pgvector cosine, top_k, threshold, user_id)
   → if no rows: return exact refusal string, do not call LLM
   → if rows: LLM answers using only those excerpts + filenames
   → persist user + assistant messages on the conversation
```

3.1 Chunking

LangChain `RecursiveCharacterTextSplitter`, `chunk_size=1000`, `chunk_overlap=200`. Empty chunks are dropped.

3.2 Embeddings

- **OpenAI:** `text-embedding-3-small`, 1536 dimensions (matches `vector(1536)` in SQL).
- **Gemini:** `gemini-embedding-001` (3072), **truncated to 1536 then L2-normalized** so it still fits the same column.

3.3 Retrieval and grounding (hallucination detection)

Postgres function `match_document_chunks`:

- Cosine distance → similarity `1 - distance`
- `ORDER BY embedding <=> query` with `IVFFlat lists = 100`
- Filters: `similarity >= threshold` (default **0.75**), `user_id`, `LIMIT top_k` (default **5**)

If the RPC returns **zero rows**, the API returns this string **exactly** and skips the LLM:

```
I couldn't find relevant information in your uploaded documents
```

That is the product's hallucination detector: **ungrounded → no generation**.

3.4 LLM (only after a match)

Prompt rule: answer only from excerpts; if excerpts are insufficient, use the same refusal sentence. Providers: Gemini `gemini-2.5-flash` (default), OpenAI `gpt-4o-mini`, Anthropic `claude-sonnet-4-5`.

3.5 Demo mode (no live Supabase)

Triggered when `SUPABASE_URL` is missing, `example.supabase.co`, or contains `your-project`. Then:

- Login: `demo@finmentor.app` / `Demo@12345` (or demo phone OTP `123456`)
- Documents, chunks, conversations live in `finmentor/db/memory_store.py`
- Indexing **does not** call embedding APIs
- Retrieval is **keyword overlap** (query tokens vs chunk text). Score `0.78 + 0.2 * overlap_ratio`, then the same `0.75` gate
- Answers are extractive quotes of matched chunks, not Gemini/OpenAI

Use demo for UI and for the metrics in §9. Production numbers need real keys + `migrations.sql`.

---

4. Architecture

```
 Browser (static/)
  index.html  styles.css  app.js
        │  Authorization: Bearer <JWT or demo.finmentor.local>
        ▼
FastAPI (finmentor/main.py)
  /auth/*          session (Supabase Auth or demo)
  /upload          OCR + insert + background index
  /query           retrieve → (LLM | refusal) → save chat
  /documents*      vault list / metadata
  /conversations*  history CRUD
  /doc-types       vault type catalog
        │
        ├─ pipeline/ocr.py
        ├─ pipeline/chunking.py
        ├─ pipeline/embeddings.py
        ├─ pipeline/retrieval.py
        ├─ db/supabase_client.py  ──► Supabase Postgres + pgvector
        │                             or db/memory_store.py (demo)
        └─ db/chat_store.py           conversations + messages
```

Auth

- Production: Supabase Auth with **anon** key (`SUPABASE_ANON_KEY`). Signup, password login, email/phone OTP, refresh, logout. `GET /documents` and `/query` require a Bearer token; `user_id` comes from the JWT, never from the client body.
- Demo: in-memory users; token `demo.finmentor.local`; user id `00000000-0000-4000-a000-000000000001`.

Data (Postgres) — see `finmentor/db/migrations.sql`, `migrations_auth.sql`, `migrations_chat.sql`:

- `documents` : id, user_id, filename, file_type, document_type, raw_text, status, timestamps
- `document_chunks` : chunk_index, chunk_text, embedding vector(1536), user_id
- `conversations` , `messages` : chat history per user

Storage of original bytes is **not** implemented; only extracted text + chunks are stored.

---

5. Repository map

Path	Role
`finmentor/main.py`	HTTP API, static files, upload/query, auth wiring
`finmentor/config.py`	Env load, fail-fast, demo flag
`finmentor/auth.py`	Signup / login / OTP / JWT
`finmentor/requirements.txt`	Python deps
`finmentor/.env.example`	Required variable names
`finmentor/Procfile`	`uvicorn finmentor.main:app`
`schemas/models.py`	Pydantic models, `DOC_TYPE_PRESETS` , `NO_MATCH_ANSWER`
`pipeline/`	OCR, chunk, embed, retrieve + LLM
`db/`	SQL migrations, Supabase client, memory + chat stores
`static/`	ChatGPT-mobile-inspired dark UI (original CSS/JS)
`samples/`	Fake Rahul Sharma corpus (eval only)
`render.yaml`	Render Web Service
`docs/DOCUMENTATION.md`	This file

Approximate application source (Python + HTML/CSS/JS + SQL, excluding docs/samples/venv): **~3,300 lines**.

---

6. HTTP API

Method	Path	Auth	Purpose
GET	`/`	no	UI
GET	`/health`	no	`{ "status": "ok" }`
GET	`/doc-types`	no	Vault type list + Custom rules
POST	`/auth/signup`	no	Email + password
POST	`/auth/login`	no	Email + password
POST	`/auth/otp`	no	Send OTP (email or phone)
POST	`/auth/verify`	no	Verify OTP
POST	`/auth/refresh`	refresh token	New access token

Method	Path	Auth	Purpose
POST	`/auth/logout`	refresh token	Sign out
POST	`/upload`	Bearer	Multipart file + `document_type`
GET	`/documents`	Bearer	Vault list
GET	`/documents/{id}`	Bearer	One document (must own it)
POST	`/query`	Bearer	`{ query, conversation_id? }`
GET	`/conversations`	Bearer	History list
POST	`/conversations`	Bearer	New empty thread
GET	`/conversations/{id}`	Bearer	Messages
DELETE	`/conversations/{id}`	Bearer	Delete thread

Upload `document_type`: one of the 19 presets below, or `custom:<label>` where label is 2–60 characters (letters, numbers, space, `& / ( ) . , _ -`).

Presets: Bank Statement, Credit Card Statement, Salary Slip, Form 16, Income Tax Return, Investment Statement, Mutual Fund Statement, Demat / Brokerage Statement, Insurance Policy, Life Insurance, Health Insurance, Loan Statement, Home Loan, Vehicle Loan, Education Loan, Rent Agreement, Utility Bill, GST Return, Other.

Query response: `answer`, `sources[]` (`document_id`, `filename`, `similarity_score`, `chunk_text`), `conversation_id`.

---

7. User interface

Single-page dark UI (`static/`):

- **Gate:** login, signup, OTP (email or phone).
- **Home:** chat composer, not a dashboard. New users see a short empty state.
- **History:** left drawer of conversations; load / delete / new chat.
- **Vault:** drawer to pick a type (including Custom) and upload. Types loaded from `GET /doc-types`.
- **Voice:** Web Speech API microphone on supporting browsers; transcript goes into the composer.
- **Sources:** each grounded assistant message lists filenames and similarity scores.

Demo login (when `SUPABASE_URL` is a placeholder):

- Email: `demo@finmentor.app`
- Password: `Demo@12345`
- Phone: `+919800000001`
- OTP: `123456`

---

8. How to run

Python 3.12+. From the repo root:

```
 python -m venv .venv
.\.venv\Scripts\activate
pip install -r finmentor/requirements.txt
copy finmentor\.env.example finmentor\.env
# fill real keys, or leave example URL for demo mode
$env:PYTHONPATH = (Get-Location).Path
uvicorn finmentor.main:app --reload --host 127.0.0.1 --port 8000
```

Open `http://127.0.0.1:8000`. Apply SQL in `finmentor/db/` on the Supabase project before using real persistence.

Deploy: `render.yaml` sets `PYTHONPATH=`.

Fake data only in this repo. Do not upload real KYC, PAN, or live bank files into shared environments.

---

9. Dummy-response evaluation (measured)

Date: 9 October 2026

Mode: Demo keyword retrieval (placeholder `SUPABASE_URL`, no embedding/LLM keys).

Corpus: three synthetic files in `samples/` (Rahul Sharma, FY 2025–26). Not real accounts.

File	Extracted chars	Chunks (size 1000 / overlap 200)
`bank_statement.txt`	747	1
`insurance_policy.txt`	576	1
`investment_record.txt`	465	1
Total	**1,788**	**3**

Index (demo, no embeddings): 2.63 ms for three files (text already extracted; OCR not used).

Queries (6): five should hit a file; one must refuse (car loan is not in the vault).

9.1 Confusion matrix (grounding decision)

Treat “returned sources” as Predicted Positive; “should be in corpus” as Actual Positive.

	Predicted match	Predicted no-match
Should match	TP = 5	FN = 0
Should refuse	FP = 0	TN = 1

Metric	Value
Accuracy	1.000
Precision	1.000

Metric	Value
Recall	1.000
F1	1.000
Source-file accuracy (top filename ∈ expected)	1.000
False-positive rate (answered with no supporting file)	0.000
Hallucinated numeric answers on the negative query	0

These scores are **on this tiny keyword corpus**. They are not production embedding/LLM quality.

9.2 Latency (server `query_documents` in-process, ms)

Query	Expected file	Similarity	Latency (ms)	Sources
How much did I spend on groceries in January?	bank_statement.txt	0.8133	3551.66	1
What does my health insurance cover?	insurance_policy.txt	0.8600	156.15	1
What's my total investment gain?	investment_record.txt	0.8300	243.09	2
What's excluded from my insurance policy?	insurance_policy.txt	0.8200	94.08	1
What was my salary credit this month?	bank_statement.txt	0.8467	36.67	1
What is my car loan outstanding balance?	*(none)*	—	0.35	0

Latency summary	ms
Mean	680.33
Median	125.12
p95	243.09
Min (true negative)	0.35
Max	3551.66

Why the first query is ~3.5 s: that run still called Supabase for filenames (`get_documents_by_ids`) while `SUPABASE_URL` was a fake host, so HTTP connect failed and was logged. Later queries were warm. Demo now reads filenames from memory so that stall should not repeat.

Typical grounded demo latency after that: tens to low hundreds of milliseconds (keyword scan + string answer). Production will add embedding RTT + pgvector + LLM RTT (usually seconds).

9.3 Dummy responses (verbatim shape)

Q1 — Groceries

Sources: `bank_statement.txt` @ 0.8133

> I found this in your uploaded files for: How much did I spend on groceries in January?
>
> From your file: Account Holder: Rahul Sharma …

Needles `4,150.00` , `grocery` : **found**. Answer 918 chars.

Q2 — Coverage

Sources: `insurance_policy.txt` @ 0.8600

Needles `Hospitalization`, `coverage`: **found**. Sum insured in the chunk: `Rs. 10,00,000`.

Q3 — Investment gain

Sources: `bank_statement.txt` and `investment_record.txt` @ 0.8300 (keyword overlap also pulled the bank statement).

Needles `15,905.83`, `8.99`: **found**.

Q4 — Exclusions

Sources: `insurance_policy.txt` @ 0.8200

Needles `Cosmetic`, `Exclusions`: **found**.

Q5 — Salary

Sources: `bank_statement.txt` @ 0.8467

Needles `85,000.00`, `Salary`: **found**.

Q6 — Car loan (negative)

Sources: none. Latency 0.35 ms.

```
I couldn't find relevant information in your uploaded documents
```

9.4 What this eval does *not* measure

- OpenAI / Gemini embedding quality or IVFFlat recall on large vaults
- LLM faithfulness (demo never called an LLM)
- OCR quality (samples are `.txt`)
- Concurrent users, JWT expiry, or Render cold start
- Token cost

Re-run in production: upload the same three files against a real project, POST `/query` with a user JWT, and compare answers + `similarity_score` + wall time.

---

10. Constraints and known limits

- Never answers from parametric knowledge when retrieval is empty.
- IVFFlat `lists = 100` is a poor fit for tiny indexes; production should tune lists as the chunk count grows, or use HNSW if the extension allows.
- Original file bytes are not stored; re-download of the PDF is impossible from this app.
- Demo keyword retrieval can attach an extra loosely related file (see Q3 bank statement). Embeddings are meant to tighten that.
- Not financial, legal, or tax advice.

---

11. Configuration reference

Copied from `finmentor/.env.example`:

```
 SUPABASE_URL
SUPABASE_KEY
SUPABASE_ANON_KEY
OCR_API_KEY
OCR_ENDPOINT
OCR_PROVIDER
LLM_API_KEY
LLM_PROVIDER          # gemini | openai | anthropic
EMBEDDING_API_KEY
EMBEDDING_PROVIDER    # openai | gemini
SIMILARITY_TOP_K      # default 5
SIMILARITY_THRESHOLD  # default 0.75
```

Dependencies: FastAPI, Uvicorn, python-dotenv, Pydantic v2, httpx, LangChain + text-splitters, supabase, openai, google-generativeai, anthropic, python-multipart.